

Website/App User Funnel Drop-off Analysis

Submitted By: Vishnuteja

```
1 # Import the required libraries
2
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6
7 # Display plots inside the notebook
8 %matplotlib inline
```

```
1 # Load the dataset
2
3 df = pd.read_csv("funnel_events_sample.csv")
4
5 # Display the first few records
6 df.head()
```

	user_id	step	timestamp
0	U102	signup_started	2026-07-20 04:17:00
1	U030	visited_site	2026-07-20 03:59:00
2	U103	signup_started	2026-07-20 03:59:00
3	U031	signup_started	2026-07-20 03:58:00
4	U122	signup_started	2026-07-20 04:10:00

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
1 print(f"Number of Rows : {df.shape[0]}")
2 print(f"Number of Columns : {df.shape[1]}")
```

Number of Rows : 542
Number of Columns : 3

Understanding the Dataset

Before performing any analysis, it is important to understand the structure of the dataset. This includes checking the number of rows and columns, identifying data types, and verifying whether there are any missing values that may affect the analysis.

```
1 # Display basic information about the dataset
2
3 print("Dataset Shape:", df.shape)
4
5 print("\nColumns:")
6 print(df.columns.tolist())
7
8 print("\nData Types:")
9 print(df.dtypes)
10
11 print("\nMissing Values:")
12 print(df.isnull().sum())
```

Dataset Shape: (542, 3)

Columns:
['user_id', 'step', 'timestamp']

Data Types:
user_id object
step object
timestamp object
dtype: object

Missing Values:
user_id 0
step 0
timestamp 0
dtype: int64

```
1 df.head()
```

	user_id	step	timestamp
56	U001	visited_site	2026-07-20 03:33:00
419	U001	signup_started	2026-07-20 04:02:00
306	U001	details_filled	2026-07-20 04:17:00
57	U002	visited_site	2026-07-20 03:34:00
174	U002	signup_started	2026-07-20 04:00:00

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
1 # Convert timestamp column to datetime format
2
3 df["timestamp"] = pd.to_datetime(df["timestamp"])
4
5 # Check for duplicate rows
6 duplicate_rows = df.duplicated().sum()
7
8 print("Duplicate Rows:", duplicate_rows)
9
10 # Remove duplicate rows if any
11 df = df.drop_duplicates()
12
13 # Verify dataset information after cleaning
14 df.info()
```

Duplicate Rows: 0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 542 entries, 0 to 541
Data columns (total 3 columns):
Column Non-Null Count Dtype
--- ---
0 user_id 542 non-null object
1 step 542 non-null object
2 timestamp 542 non-null datetime64[ns]
dtypes: datetime64[ns](1), object(2)
memory usage: 12.8+ KB

```
1 # Define the funnel stages in the correct order
2
3 funnel_steps = [
4     "visited_site",
5     "signup_started",
6     "details_filled",
7     "email_verified",
8     "purchase_completed"
9 ]
10
11 # Convert the step column into an ordered categorical variable
12 df["step"] = pd.Categorical(
13     df["step"],
14     categories=funnel_steps,
15     ordered=True
16 )
17
18 # Sort the dataset based on user and funnel stage
19 df = df.sort_values(["user_id", "step"])
20
21 # Preview the cleaned dataset
22 df.head()
```

	user_id	step	timestamp
56	U001	visited_site	2026-07-20 03:33:00
419	U001	signup_started	2026-07-20 04:02:00
306	U001	details_filled	2026-07-20 04:17:00
57	U002	visited_site	2026-07-20 03:34:00
174	U002	signup_started	2026-07-20 04:00:00

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
1 # Calculating Funnel Metrics Count unique users at each funnel stage
2 funnel_counts = (
3     df.groupby("step")["user_id"]
```

```

4     .nunique()
5     .reindex(funnel_steps)
6 )
7 funnel_df = pd.DataFrame({
8     "Stage": funnel_counts.index,
9     "Unique Users": funnel_counts.values
10 })
11
12 funnel_df

```

/tmp/ipykernel_1367/4173528166.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True
df.groupby("step")["user_id"]

	Stage	Unique Users
0	visited_site	200
1	signup_started	150
2	details_filled	96
3	email_verified	52
4	purchase_completed	44

Next steps: [Generate code with funnel_df](#) [New interactive sheet](#)

```

1 # Calculate conversion rate and drop-off
2
3 conversion_rates = [100]
4 drop_off_users = [0]
5
6 for i in range(1, len(funnel_df)):
7     previous = funnel_df.loc[i-1, "Unique Users"]
8     current = funnel_df.loc[i, "Unique Users"]
9
10    conversion = (current / previous) * 100
11    conversion_rates.append(round(conversion, 2))
12
13    drop_off_users.append(previous - current)
14
15 funnel_df["Conversion Rate (%)"] = conversion_rates
16 funnel_df["Users Dropped"] = drop_off_users
17
18 funnel_df

```

	Stage	Unique Users	Conversion Rate (%)	Users Dropped
0	visited_site	200	100.00	0
1	signup_started	150	75.00	50
2	details_filled	96	64.00	54
3	email_verified	52	54.17	44
4	purchase_completed	44	84.62	8

Next steps: [Generate code with funnel_df](#) [New interactive sheet](#)

```

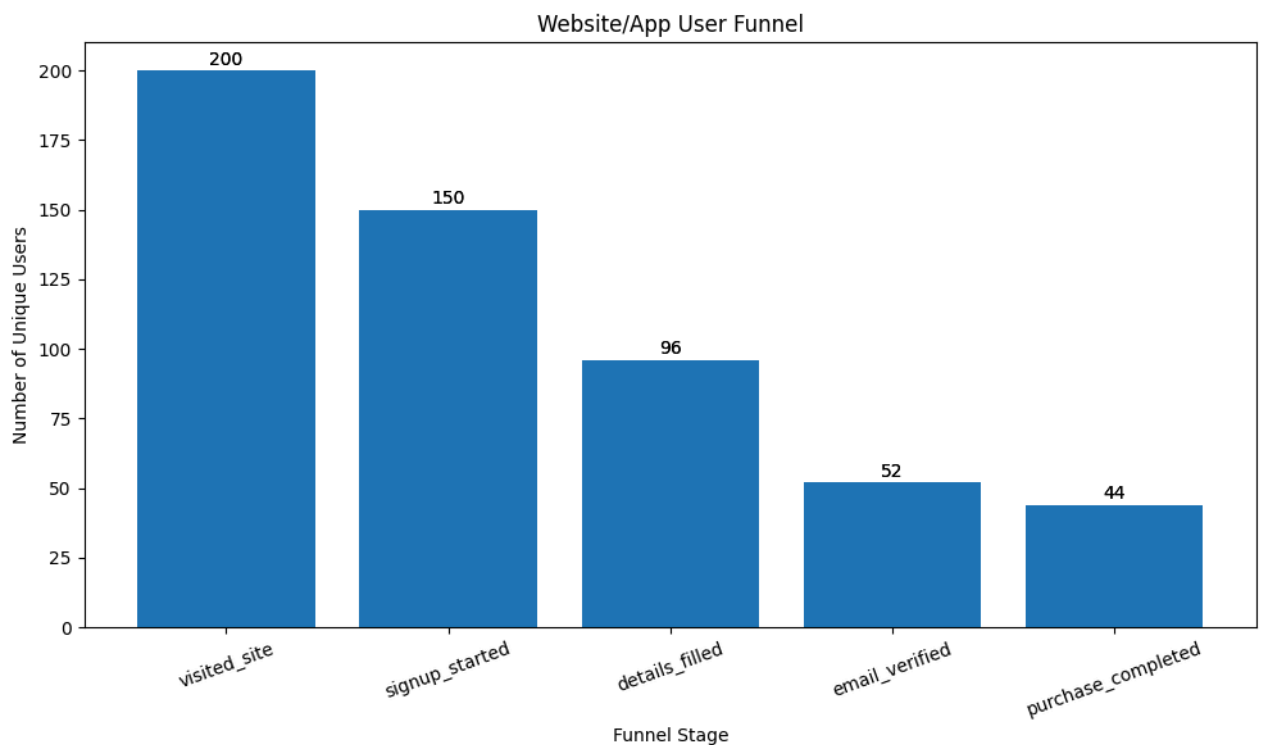
1 # Visualize the funnel stages
2
3 plt.figure(figsize=(10,6))
4 bars = plt.bar(
5     funnel_df["Stage"],
6     funnel_df["Unique Users"]
7 )
8
9 for bar in bars:
10    height = bar.get_height()
11    plt.text(
12        bar.get_x()+bar.get_width()/2,
13        height+2,
14        int(height),
15        ha="center"
16    )
17
18
19 plt.title("Website/App User Funnel")
20 plt.xlabel("Funnel Stage")
21 plt.ylabel("Number of Unique Users")
22

```

```

23 plt.xticks(rotation=20)
24
25 for index, value in enumerate(funnel_df["Unique Users"]):
26     plt.text(index, value + 2, str(value), ha='center')
27
28 plt.tight_layout()
29

```



```

1 # Find the stage with the maximum drop-off
2
3 dropoff = funnel_df.iloc[1:].copy()
4
5 largest_drop = dropoff.loc[dropoff["Users Dropped"].idxmax()]
6
7 previous_stage = funnel_df.loc[
8     largest_drop.name - 1,
9     "Stage"
10 ]
11
12 print(f"Largest Drop-off Stage : {previous_stage} → {largest_drop['Stage']}")
13 print(f"Users Dropped          : {largest_drop['Users Dropped']}")
14 print(f"Conversion Rate         : {largest_drop['Conversion Rate (%)']:.2f}%")

```

```

Largest Drop-off Stage : signup_started → details_filled
Users Dropped          : 54
Conversion Rate         : 64.00%

```

```

1 # Calculate average time between consecutive stages
2
3 df_sorted = df.sort_values(["user_id", "timestamp"])
4
5 pivot = df_sorted.pivot_table(
6     index="user_id",
7     columns="step",
8     values="timestamp",
9     aggfunc="first"
10 )
11
12 print("Average Time Between Consecutive Stages\n")
13
14 for i in range(len(funnel_steps)-1):
15
16     current = funnel_steps[i]
17     nxt = funnel_steps[i+1]
18
19     if current in pivot.columns and nxt in pivot.columns:
20
21         diff = (
22             pivot[nxt] - pivot[current]
23         ).dropna()

```

```
24
25     if len(diff) > 0:
26         avg = diff.mean()
27
28     print(
29         f"{current} → {nxt} : {avg}"
30     )
```

Average Time Between Consecutive Stages

visited_site → signup_started : 0 days 00:15:14.400000
signup_started → details_filled : 0 days 00:18:26.086956521
/tmp/ipykernel_1367/3505674466.py:5: FutureWarning: The default value of observed=False is deprecated and will change to obs
pivot = df_sorted.pivot_table()

```
1 print("Final Funnel Summary")
2
3 funnel_df
```

Final Funnel Summary

	Stage	Unique Users	Conversion Rate (%)	Users Dropped
0	visited_site	200	100.00	0
1	signup_started	150	75.00	50
2	details_filled	96	64.00	54
3	email_verified	52	54.17	44
4	purchase_completed	44	84.62	8

Next steps: [Generate code with funnel_df](#) [New interactive sheet](#)

Observations

- 200 unique users visited the website.
- 150 users initiated the signup process.
- The highest user drop-off occurred between the Details Filled and Email Verified stages.
- Only 44 users successfully completed the purchase.

Conclusion

This analysis successfully calculated the conversion rate at each stage of the signup funnel and identified the largest drop-off point. The highest user loss occurred between the Details Filled and Email Verified stages, indicating that improving the email verification process could increase the overall funnel conversion.